网页文章错误检测 Agent —— 详细设计文档

> **项目**：网页文章错误检测 Agent（浏览器插件）
> **课程**：软件工程与计算III · 迭代三
> **文档定位**：迭代三详细设计，涵盖模块分解、接口定义、数据库表结构、事件模型和前端组件设计。需求背景参见产品化需求分析文档。

1. 设计概览

1.1 迭代三模块文件清单

```
agent/
├── agent/
│   ├── agent.py          [修改]  — 主控辩论流 + 聊天功能
│   ├── debate.py         [新增]  — 辩论提示词/解析/总结/claim 结果构建
│   ├── models.py         [修改]  — 新增 DebateEvent/SummaryEvent/ChatEvent/ChatChunkEvent/ChatDoneE
│   ├── scanner.py        [修改]  — 声明筛选增强（排除规则）
│   ├── planner.py        [不变]
│   ├── preprocess.py     [不变]
│   ├── dataset_loader.py [不变]
│   ├── llm/
│   │   ├── base.py        [修改]  — 新增 complete_stream()
│   │   ├── claude.py      [修改]  — 实现流式
│   │   ├── glm.py         [不变]
│   │   └── chat_factory.py [新增] — create_chat_llm()
│   ├── skills/           [新增]
│   │   ├── __init__.py
│   │   ├── base.py        — Skill 数据结构、load_skills()、build_overlay_skill()
│   │   ├── router.py      — route_skill() GLM-4-Flash 领域路由
│   │   └── defs/          — 内置 Skill markdown 文件
│   │       ├── general.md
│   │       ├── medical.md
│   │       ├── finance.md
│   │       ├── technology.md
│   │       └── news_policy.md
│   └── tools/
│       ├── registry.py    [修改]  — ToolSpec 新增 domains 字段，14 个工具
│       ├── web_search.py
│       ├── wikipedia_lookup.py
│       ├── source_verifier.py
│       ├── cross_reference.py
│       ├── wikidata_lookup.py    [新增]
│       ├── official_statistics.py [新增]
│       ├── stock_quotes.py       [新增]
│       ├── pubmed_search.py      [新增]
│       ├── consumer_health.py    [新增]
│       ├── academic_search.py    [新增]
│       ├── arxiv_search.py       [新增]
│       ├── patent_lookup.py      [新增]
│       └── fact_check.py         [新增]
├── backend/
│   ├── main.py           [修改]  — SSE 持久化、chat_stream、heartbeat、/skills
│   ├── dependencies.py   [新增]  — get_db、get_device_id
│   ├── db/
│   │   ├── __init__.py
```

```
│   │   ├── init_db.py     ── Base.metadata.create_all()
│   │   ├── session.py     ── SessionLocal
│   │   └── models.py      ── 7 张表 ORM
│   ├── routers/
│   │   ├── __init__.py
│   │   └── sessions.py    ── 9 个 REST 端点
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── session.py     ── SessionCreate / SessionResponse
│   │   ├── event.py
│   │   ├── claim.py       ── ClaimDetailResponse / ToolCallItem
│   │   ├── chat.py        ── ChatSend / ChatSendResponse
│   │   └── common.py
│   └── services/
│       ├── __init__.py
│       ├── session_service.py
│       ├── event_service.py
│       ├── chat_service.py
│       └── context_builder.py  ── build_chat_context()
```

1.2 事件类型总览

事件	type	说明
PlanEvent	plan	扫描完成，声明列表（不变）
ThinkingEvent	thinking	ReAct 每步思考（不变）
ToolCallEvent	tool_call	工具调用及结果（不变）
AnnotationEvent	annotation	单条声明最终标注（不变）
DebateEvent	debate	**新增**：统一辩论事件（三阶段四角色）
SummaryEvent	summary	**新增**：分析总结
ChatEvent	chat	**新增**：对话消息
ChatChunkEvent	chat_chunk	**新增**：流式对话 token
ChatDoneEvent	chat_done	**新增**：流式对话结束
StatusEvent	status	阶段状态（含 heartbeat）
ErrorEvent	error	异常（不变）
DoneEvent	done	全完成（扩展 total_claims/claim_results/summary_available/reverify_supported）

2. Agent 核心详细设计

2.1 Agent 类设计（agent/agent/agent.py）

```
class Agent:
    def __init__(self, complex_llm, router_llm=None, chat_llm=None):
        self._llm = complex_llm          # Claude，ReAct 推理
        self._router_llm = router_llm    # GLM-4-Flash，领域分类
        self._chat_llm = chat_llm        # 聊天专用 LLM
        self._skills = load_skills()     # 启动时加载所有 defs/*.md

    async def run(self, article_text: str,
                  overlays: list[dict] | None = None,
                  disabled_tools: list[str] | None = None):
        # 并行验证 (Semaphore 3) + Queue 汇聚 + 辩论流

    async def chat(self, session_context: dict,
                   user_message: str,
                   history: list[dict] | None = None):
        # 基于会话上下文的流式追问回答
```

2.2 主流程（Agent.run()）

```
1. scan_article(text, self._llm) → claims
2. build_plan(claims) → VerificationPlan
3. prepare_cross_reference_context(text)
4. emit PlanEvent
5. route_skill(article, skills, router_llm) → skill
6. 处理 disabled_tools → effective_skill（做减法）
7. 处理 overlays → active_overlays（build_overlay_skill 校验）
8. Semaphore(3) 并行处理 claims:
   a. _debate_claim(claim) → 辩论流
   b. 事件 via asyncio.Queue 汇聚后 SSE yield
9. build_summary_event(claim_records) → SummaryEvent
10. emit DoneEvent(total_annotations, claim_results=...)
```

2.3 辩论引擎（agent/agent/debate.py）

核心数据结构：

结构	用途
`VerificationRecord`	单次 Verifier 执行结果：annotation + tool_calls + thoughts + errors
`ChallengeRecord`	Challenger 审查结果：stance + confidence + reasoning + missing_evidence + suggested_queries

结构	用途
`ClaimDebateRecord`	单条声明的完整辩论记录：claim + skill_name + verifier + challenger + judge_annotation + rebuttal

核心函数：

函数	输入	输出
`build_challenger_prompt()`	claim_text, skill_name, verifier_record	Challenger 审查提示词
`parse_challenger_response()`	LLM raw text	ChallengeRecord
`build_rebuttal_instruction()`	challenge_record	重验证指令（注入 ReAct）
`build_judge_prompt()`	claim_text, skill_name, verifier, challenger, rebuttal	Judge 裁决提示词
`parse_judge_response()`	raw, claim, fallback_annotation	(AnnotationEvent, debate_summary)
`build_summary_event()`	list[ClaimDebateRecord]	SummaryEvent
`build_claim_result()`	ClaimDebateRecord	dict（供前端持久化）

JSON 解析：全部使用 `json_repair` 库（`json_repair.loads()`），修复单引号、尾逗号、中文标点等常见 LLM 输出错误。

2.4 辩论流程（`Agent._debate_claim()`）

```
async def _debate_claim(self, claim, skill, overlays, record_sink, disabled_note_tools):
    # 1. 辩论开始
    yield DebateEvent(phase="started", role="coordinator")

    # 2. Verifier ReAct（emit_annotation=False，结果写入 verifier_sink）
    async for event in self._react_loop(claim, skill, overlays, ...):
        yield event
    verifier_record = self._build_verification_record(claim, "verifier", verifier_sink)
    yield DebateEvent(phase="argument", role="verifier", ...)

    # 3. 高置信快速通道
    if verifier_record.annotation.confidence >= 0.9 and not verifier_record.errors:
        # 跳过 Challenger/Judge，直接校准并输出
        yield DebateEvent(phase="result", role="verifier", ...)
        yield self._calibrate_annotation(verifier_record.annotation, verifier_record)
        return

    # 4. Challenger 审查
    challenge = await self._run_challenger(claim.text, skill.name, verifier_record)
    yield DebateEvent(phase="argument", role="challenger", stance=challenge.stance, ...)

    # 5. Challenger 异议 → 重验证
    if challenge.stance == "challenge":
        async for event in self._react_loop(claim, skill, overlays,
            extra_instruction=build_rebuttal_instruction(challenge), ...):
            yield event
        rebuttal_record = self._build_verification_record(claim, "verifier_rebuttal", ...)

    # 6. Judge 裁决
    judge_annotation, debate_summary = await self._run_judge(...)
    calibrated = self._calibrate_annotation(judge_annotation, verifier_record, rebuttal_record)
    yield DebateEvent(phase="result", role="judge", ...)
    yield calibrated
```

2.5 置信度校准（`Agent._calibrate_annotation()`）

根据核查过程的客观信号对 LLM 自评置信度做乘法修正：

信号	系数
未调任何工具	×0.80
有工具/格式错误	×0.85
无证据 URL	×0.90
步数 ≥ 5	×0.90

信号	系数
使用 ≥ 2 种不同工具	×1.10

校准后 `confidence < 0.60` 且 `error_type != None` → 忽略该发现（`error_type = None`）。

2.6 聊天功能（`Agent.chat()`）

```
async def chat(self, session_context, user_message, history=None):
    # 1. _build_chat_system_prompt(session_context) → system_prompt
    #    - 包含文章标题 + 正文(截断3000字) + 声明验证结果 + 总结
    # 2. messages = [system] + history(最近20条) + [user_message]
    # 3. for token in self._chat_llm.complete_stream(messages):
    #        yield token
```

3. Skill 系统详细设计

3.1 Skill 文件格式（`agent/skills/defs/*.md`）

```
---
name: medical
description: 医学健康类文章的核查—涉及药物、疾病、临床试验等声明
allowed_tools: [pubmed_scientific_search, consumer_health_verifier, web_search, cross_reference]
kind: domain
---
你是专业的医学事实核查 Agent。
优先查阅 PubMed、WHO、FDA 等权威医学来源。
对药物剂量声明严格数值比对...
```

3.2 Skill 数据类（`agent/skills/base.py`）

```
@dataclass(frozen=True)
class Skill:
    name: str                       # 唯一标识
    description: str                # 路由触发描述（给路由模型看）
    prompt: str                     # 正文：核查指令，注入 ReAct system prompt
    allowed_tools: tuple[str, ...]  # 工具白名单
    kind: str = "domain"            # "domain" | "overlay"
```

3.3 Skill 加载与校验

```
def load_skills(skill_dir=None) -> dict[str, Skill]:
    # 1. 扫描 defs/ 目录下所有 *.md
    # 2. 解析 YAML frontmatter + 正文
    # 3. 校验：必须包含 general 兜底 skill
    # 4. 校验：每个 skill 的 allowed_tools 都在 TOOL_REGISTRY 中
    # 5. 返回 {name: Skill}
```

3.4 Overlay 构造（`build_overlay_skill()`）

```
def build_overlay_skill(data: dict) -> Skill:
    # 从用户(浏览器插件)传来的 JSON 构造 overlay Skill
    # kind 固定为 "overlay"
    # 不接受 allowed_tools（工具边界由 domain skill 决定）
    # name ≤ 40 字符，prompt ≤ 4000 字符
    # 校验失败抛 ValueError
```

3.5 Skill 注入 System Prompt（`Agent._build_system_prompt()`）

```
最终 System Prompt =
    "你是 Verifier Agent..."  (基础指令)
    ↑
    skill.prompt               (领域核查要点)
    ↑
    overlay.prompt × N          (附加关注点，可多个)
    ↑
    已禁用工具提示               (如果 disabled_note_tools 非空)
    ↑
    可用工具列表                 (skill.allowed_tools)
    ↑
    输出格式规则                 (固定 ReAct 格式)
```

4. 工具注册表详细设计

4.1 ToolSpec (agent/tools/registry.py)

```
@dataclass
class ToolSpec:
    name: str
    description: str        # 给 LLM 看，影响 Agent 选工具
    input_schema: dict      # JSON Schema
    func: Callable          # async def func(input: str) -> str
    domains: list[str]      # 适用领域（新增）
```

4.2 工具与领域映射

```
general (兜底):  web_search, wikipedia_lookup, source_verifier, cross_reference,
                 wikidata_lookup, fact_check_domestic, fact_check_global
medical:          pubmed_scientific_search, consumer_health_verifier
finance:          macro_statistics_global, stock_market_quotes
technology:       academic_paper_search, preprint_arxiv_search, patent_status_lookup, wikidata_lookup
news_policy:      fact_check_domestic, fact_check_global
```

4.3 工具调用守卫 (Agent._react_loop())

```
if tool_name not in TOOL_REGISTRY:
    → "工具不存在，请换用可用工具"
elif tool_name in disabled_note_tools:
    → "工具已被用户禁用"
elif tool_name not in skill.allowed_tools:
    → "当前领域不允许使用此工具，可用：{allowed}"
else:
    → 执行工具调用
```

5. 后端 API 详细设计

5.1 接口总览

方法	路径	说明
POST	/analyze	核心分析（SSE），新增 overlays、disabled_tools 参数

方法	路径	说明
GET	/skills	Skill + 工具目录（含 used_by 反查）
POST	/api/v1/sessions/{id}/chat/stream	SSE 流式对话
POST	/api/v1/sessions	创建会话
GET	/api/v1/sessions	会话列表
GET	/api/v1/sessions/{id}	会话详情
GET	/api/v1/sessions/{id}/events	事件历史
GET	/api/v1/sessions/{id}/claims/{claim_id}	单条声明详情
GET	/api/v1/sessions/{id}/summary	会话总结
POST	/api/v1/sessions/{id}/chat	发送追问
GET	/api/v1/sessions/{id}/chat	对话历史
POST	/api/v1/sessions/{id}/recheck	提交重验证
GET	/health	健康检查

5.2 POST /analyze 变更

```
Request:
{
    "article_text": "string (required)",
    "article_title": "string (optional)",
    "source_url": "string (optional)",
    "overlays": [{"name": "...", "prompt": "...", "description": "..."}]  (optional),
    "disabled_tools": ["tool_name", ...]  (optional)
}

Response: SSE Stream (同迭代二 + debate/summary/chat_chunk 事件)

Request Headers:
    X-Device-ID: 匿名设备标识（可选）
```

5.3 POST /api/v1/sessions/{id}/chat/stream

```
Request:
{
    "message": "用户追问 (required)",
    "related_claim_id": null (optional),
    "mode": null (optional)
}

Response: SSE Stream
event: chat_chunk   → {"type":"chat_chunk","content":"你","message_id":"xxx"}
event: chat_chunk   → {"type":"chat_chunk","content":"好","message_id":"xxx"}
...
event: chat_done    → {"type":"chat_done","message_id":"xxx","session_id":"yyy","full_content":"..."}
```

5.4 GET /skills

```
Response:
{
    "domains": [{"name": "medical", "description": "...", "kind": "domain"}, ...],
    "tools": [
        {
            "name": "pubmed_scientific_search",
            "description": "搜索 PubMed...",
            "used_by": ["medical"]
        },
        ...
    ],
    "overlay_limits": {"max_overlays_per_request": 10, "max_name_len": 40, "max_prompt_len": 4000},
    "disabled_tools_limits": {"max_disabled_tools_per_request": 50}
}
```

6. 事件模型详细设计

6.1 新增事件（`agent/agent/models.py`）

DebateEvent —— 统一辩论事件：

```python
class DebateEvent(BaseModel):
    type: Literal["debate"] = "debate"
    claim_id: str
    round: int                              # 辩论轮次（1 或 2）
    phase: Literal["started", "argument", "result"]
    role: Literal["coordinator", "verifier", "challenger", "judge"]
    message: str
    stance: Literal["support", "challenge", "neutral"] = "neutral"
    confidence: float | None = None
    evidence_urls: list[str] = []
    details: dict[str, Any] = {}            # {"fast_path": bool, "reverify_target": str, ...}
```

SummaryEvent：

```python
class SummaryEvent(BaseModel):
    type: Literal["summary"] = "summary"
    total_claims: int
    total_annotations: int
    clean_claims: int
    challenged_claims: int
    revised_claims: int
    error_breakdown: dict[str, int]          # {"factual_error": 2, ...}
    representative_claims: list[dict]
    overall_conclusion: str
    reverify_supported: bool = True
```

ChatEvent / ChatChunkEvent / ChatDoneEvent：

```python
class ChatChunkEvent(BaseModel):
    type: Literal["chat_chunk"] = "chat_chunk"
    content: str                             # 单个 token
    message_id: str | None = None

class ChatDoneEvent(BaseModel):
    type: Literal["chat_done"] = "chat_done"
    message_id: str
    session_id: str
    full_content: str | None = None
```

6.2 DoneEvent 扩展

```python
class DoneEvent(BaseModel):
    type: Literal["done"] = "done"
    total_annotations: int
    total_claims: int = 0                    # 新增
    summary_available: bool = False          # 新增
    reverify_supported: bool = True          # 新增
    claim_results: list[dict[str, Any]] = [] # 新增：每条声明的完整辩论结果
```

7. 数据库详细设计

7.1 表结构（backend/db/models.py）

7 张表全部使用 SQLAlchemy ORM，Text 列存储 ISO 8601 时间戳字符串：

表	主键	外键	记录数预估（单次分析）
analysis_session	sess_xxx	—	1
claim_record	claim_xxx	session_id	5-15 条
event_record	evt_xxx	session_id	30-80 条
tool_call_record	tool_xxx	session_id, claim_record_id	5-30 条
summary_record	sum_xxx	session_id (unique)	1
chat_message	msg_xxx	session_id	0-50 条
skill_record	skill_xxx	—	固定 5 条

7.2 事件持久化（_persist_agent_event()）

```
dispatch by event_type:
  plan       → 创建 ClaimRecord × N, 更新 AnalysisSession.status=running
  annotation → 更新 ClaimRecord (error_type, confidence, reasoning, evidence_urls, verdict)
  tool_call  → 写入 ToolCallRecord
  debate     → 写入 EventRecord, phase=result 时更新 ClaimRecord
  summary    → 写入/更新 SummaryRecord
  done       → 更新 AnalysisSession.status=completed, 兜底构建 SummaryRecord
  status     → stage=route 时回填 skill_id 和 domain
  error      → 无 claim_id 时更新 AnalysisSession.status=failed
  default    → 写入 EventRecord
```

7.3 初始化（backend/db/init_db.py）

```
def init_db():
    Base.metadata.create_all(bind=engine)
```

8. 前端设计要点

8.1 Side Panel 三区布局

8.2 Popup 界面

- 技能选择（下拉菜单，自动路由也可手动选）
- 工具勾选列表（从 `/skills` 接口获取，含 `used_by` 反查提示影响范围）
- 附加视角 (overlay) 添加（名称 + prompt 输入）
- 开始扫描按钮

8.3 辩论事件前端渲染

`DebateEvent` 按 `phase` 和 `role` 区分展示：

- `phase=started`：辩论轮次标记
- `phase=argument role=verifier`：蓝色边框，展示初判 + 置信度 + 工具调用次数
- `phase=argument role=challenger`：橙色/黄色边框，展示立场 (support/challenge) + 缺失证据
- `phase=result`：绿色边框（Judge 裁决），展示终裁